Practice · Two-dimensional arrays

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · Declare a table

Which line makes a table of `int` with 3 rows and 5 columns?

- `int t[5][3];`
- **Right answer:** `int t[3][5];`
- `int t[3, 5];`
- `int t[15];`

Feedback: The first number is the rows, the second the columns: `int t[3][5];`. `int t[5][3];` has 5 rows and 3 columns. `int t[15];` has 15 boxes too, but in one row, not a table. Class 13, slides 9 and 10.

(mc) L1 · Which number is the row?

In `t[2][4]`, which number is the row?

- **Right answer:** `2`: the row always comes first.
- `4`: the column comes first.
- Both: the box is in row 2 and in row 4.
- It depends on how the table was declared.

Feedback: `t[row][column]`, always in that order. `t[2][4]` is row 2, column 4. Class 13, slides 9 and 15.

(mc) L1 · An initializer list

Which list fills a 2 by 2 table with `1 2` in row 0 and `3 4` in row 1?

- `int t[2][2] = {{1, 3}, {2, 4}};`
- `int t[2][2] = {1, 2}, {3, 4};`
- `int t[2][2] = {{1, 2}}, {{3, 4}};`
- **Right answer:** `int t[2][2] = {{1, 2}, {3, 4}};`

Feedback: One pair of braces around the whole list, and one pair inside it for each row, with a comma between the rows. `{{1, 3}, {2, 4}}` puts 1 and 3 in row 0. Class 13, slide 11.

(mc) L2 · Change one box

```
int t[2][2] = {{5, 6}, {7, 8}};
t[1][0] = t[0][1] + 1;
printf("%d %d\n", t[1][0], t[1][1]);
```

What does it print?

- `8 8`
- **Right answer:** `7 8`
- `6 8`
- `7 6`

Feedback: `t[0][1]` is row 0, column 1: 6. So `t[1][0]` becomes 7. `t[1][1]` is still 8. Class 13, slides 10 to 13.

(mc) L2 · A table of zeros

`int z[3][3] = {0};` What is in `z[2][2]`?

- Garbage: only the first box is set.
- `3`
- **Right answer:** `0`
- It does not compile: the list is too short.

Feedback: Every box that the list does not give a value becomes 0. `{0}` gives the first box 0, and all the others become 0 too. So the whole table is 0. Class 13, slide 16.

(mc) L2 · A board of char

```
char g[2][3];
for (int r = 0; r < 2; r++) {
    for (int c = 0; c < 3; c++) {
        g[r][c] = '.';
    }
}
g[0][1] = '*';
```

Two more loops print `g` with `%c`, one row per line. What is on the screen?

- **Right answer:** `.*.` then `...`
- `...` then `.*.`
- `*..` then `...`
- `.*` then `..` then `..`

Feedback: Every box gets a dot first. Then `g[0][1]`, row 0 and column 1, gets the star: the middle of the first row. The table has 2 rows of 3. Class 13, slide 41.

(fib) L1 · How many boxes

A table of `int` has 5 rows and 7 columns. How many boxes does it hold? Type only the number.

Boxes: [35]

Feedback: 5 rows times 7 columns is 35 boxes. Class 13, slide 9.

(fib) L1 · The last row

A table has 4 rows. What is the index of its last row? Type only the number.

Last row: [3]

Feedback: Indexes start at 0, so 4 rows are rows 0, 1, 2 and 3. The last row is always the number of rows minus 1. Class 13, slides 9 and 40.

(fib) L2 · A box with no value

An initializer list gives values to some boxes of a table and not to others. What value does a box get when the list gives it none? Type only the number.

Value: [0]

Feedback: Every box the list does not mention becomes 0. That is why `= {0}` makes a whole table 0. A table with no list at all is different: its boxes hold garbage. Class 13, slide 16.